

Improving MonoGS: View-Direction-Aware Regularization and Historical Keyframe Replay for Gaussian Splatting SLAM

Jiayu Lin

University of Shanghai for Science and Technology

Shanghai, China

2335052103@st.usst.edu.cn

Abstract—MonoGS pioneers dense SLAM using 3D Gaussian Splatting (3DGS), achieving real-time tracking and photorealistic mapping via differentiable tile-based rasterization. Despite its strong performance, we identify two independent methodological defects in the original design that limit its accuracy and robustness. First, the isotropic regularization penalizes all anisotropic stretching uniformly, thereby suppressing geometrically efficient surface-parallel elongation while insufficiently penalizing harmful view-aligned stretching that causes screen-space ill-conditioning. Second, the sliding-window mapping backend ceases to supervise Gaussians observed only by evicted keyframes, leading to catastrophic forgetting in the map and elevated rendering residuals for historical regions. To address these, we propose (1) view-direction-aware isotropic regularization (B4), which modulates the anisotropy penalty by the alignment between a Gaussian’s major axis and the camera viewing direction, and (2) historical keyframe replay (D2), which randomly replays evicted frames at a 10R. Rigorous ablation studies on TUM RGB-D and Replica datasets demonstrate that D2 consistently improves tracking (ATE 3.28 cm vs. 3.45 cm baseline, -4.9%). Crucially, the combined Full system exhibits negative interference, with performance dropping below the baseline on RGB-D sequences, which we analyze through the lens of gradient landscape incompatibility. Our findings highlight that the primary bottleneck in Gaussian Splatting SLAM lies in the spatio-temporal distribution of supervision signals, rather than in resource allocation or densification schedules.

Index Terms—SLAM, 3D Gaussian Splatting, differentiable rendering, isotropic regularization, catastrophic forgetting, keyframe management.

I. INTRODUCTION

Dense simultaneous localization and mapping (SLAM) is a cornerstone of robotics and augmented reality, demanding both geometric accuracy for camera tracking and photorealistic fidelity for downstream applications. The advent of Neural Radiance Fields (NeRF) [3] demonstrated that implicit scene representations could achieve unprecedented novel-view synthesis quality. However, the volumetric rendering integral required by NeRFs is computationally prohibitive for real-time SLAM systems, necessitating hierarchical encoding structures or deferred rendering pipelines that compromise fine geometric detail.

3D Gaussian Splatting (3DGS) [2] introduced a paradigm shift by representing scenes as an explicit set of anisotropic 3D Gaussians optimized via differentiable tile-based rasterization.

This explicit representation avoids costly ray marching, enabling real-time rendering while preserving competitive image quality. MonoGS [1] was the first to extend 3DGS to the SLAM setting, proposing a monocular-to-RGB-D framework that jointly optimizes camera poses and a dense Gaussian map through a frontend-backend architecture. The frontend performs frame-to-model tracking and keyframe selection, while the backend refines Gaussian parameters ($\mu, \mathbf{q}, \mathbf{s}, \mathbf{c}, \alpha$) over a sliding window of active keyframes.

Motivation. Despite MonoGS’s impressive results, its design contains under-explored assumptions that leave performance headroom unclaimed. Through systematic reproduction and diagnosis (Sec. III–IV), we identify two *independent* methodological defects:

- 1) **Geometrically blind regularization.** MonoGS employs a uniform isotropic loss that pushes all Gaussians toward spheres. Yet anisotropy is not uniformly harmful: elongation parallel to surfaces is an efficient approximation strategy, whereas stretching along the viewing direction causes severe screen-space aliasing and novel-view artifacts. A uniform penalty cannot distinguish these cases (Sec. IV-A).
- 2) **Catastrophic forgetting in the Gaussian map.** Once a keyframe is evicted from the sliding window, its observed spatial regions receive no further direct gradient signal. We empirically show that evicted-frame residuals exhibit a persistent upward trend relative to in-window frames and remain on average $1.3\text{--}1.8\times$ higher in the latter half of training (iterations $>12,000$), indicating progressive map degradation in regions no longer directly supervised (Sec. IV-B).

Contributions. We propose two targeted, methodology-level improvements:

- We introduce **view-direction-aware isotropic regularization** (B4) that weights the anisotropy penalty by the cosine alignment $\alpha = |\mathbf{v} \cdot \mathbf{d}|$ between the camera viewing direction $\mathbf{v}$ and the Gaussian’s major axis $\mathbf{d}$. This preserves efficient surface-parallel stretching while aggressively penalizing view-aligned artifacts that induce screen-space ill-conditioning.
- We introduce **historical keyframe replay** (D2), a simple

yet effective mechanism that samples evicted keyframes during mapping iterations with a 10% probability and blends their photometric loss into the optimization objective with a weight of 0.5. This provides continued direct supervision to historical regions without destabilizing the active tracking window.

We evaluate both improvements on the TUM RGB-D [13] and Replica [14] datasets under strictly controlled conditions: identical hardware, software versions, hyperparameters, and random seeds. Our ablation study reveals that D2 delivers consistent gains across all metrics, while B4 fails to improve tracking accuracy and degrades perceptual quality (LPIPS), revealing that view-aware penalty alone is insufficient without a learnable weighting function. Strikingly, the combined Full system underperforms the individual modules, exposing a negative interference phenomenon that we attribute to incompatible gradient landscapes.

II. RELATED WORK

Neural Implicit SLAM. Early dense SLAM systems relied on explicit representations such as truncated signed distance functions (TSDF) [11] or surfels [12]. The rise of coordinate-based neural networks enabled implicit mapping, with iMAP [5] and NICE-SLAM [6] demonstrating scalable indoor reconstruction. Point-SLAM [4] introduced a neural point-cloud representation with learned descriptors, improving rendering fidelity over pure MLPs. However, these methods require minutes to hours per scene due to volumetric rendering or MLP query overhead, precluding real-time operation.

3D Gaussian Splatting for SLAM. 3DGS [2] represents scenes as millions of anisotropic 3D Gaussians rasterized via splatting, achieving 100+ FPS rendering. MonoGS [1] adapted this representation to SLAM, coupling differentiable rendering with a keyframe-based backend. Subsequent works such as GS-SLAM [8] and SplatAM [9] have explored depth-guided densification and tracking strategies. While these methods advance the state of the art, they largely inherit MonoGS’s backend design, including its regularization and keyframe management policies.

Regularization in 3DGS. The original 3DGS paper does not constrain Gaussian anisotropy, which can lead to elongated “spider-web” artifacts in unbounded scenes. MonoGS introduced an isotropic regularization $\mathcal{L}_{\text{iso}} = \lambda \sum_i \|\mathbf{s}_i - \bar{\mathbf{s}}_i \mathbf{1}\|_1$ with $\lambda = 10$ to encourage compact, spherical Gaussians. However, this uniform penalty is geometrically agnostic: it penalizes surface-parallel elongation that is, in fact, a desirable compression strategy for planar regions. No prior work in Gaussian Splatting SLAM has investigated view-dependent regularization strategies.

Keyframe Management and Forgetting. Keyframe-based SLAM systems [7] traditionally maintain geometric constraints for all landmarks via a factor graph, ensuring that evicted keyframes still influence the map through shared 3D points. In differentiable SLAM, constraints are implicit: they exist only if a frame is rendered and back-propagated. Consequently, when MonoGS evicts a keyframe, its geometric

and photometric constraints vanish entirely. This phenomenon is analogous to catastrophic forgetting in continual learning, yet it remains unaddressed in the context of Gaussian-based SLAM.

III. BASELINE REPRODUCTION

A. Experimental Protocol

We reproduce MonoGS on two scenes following the official evaluation protocol: (1) **TUM RGB-D fr3/office** [13] (monocular, 2585 frames, 640×480), a challenging real-world sequence with low-texture regions, dynamic objects, and photometric variation; and (2) **Replica room0** [14] (RGB-D, 2000 frames, 1200×680), a high-fidelity synthetic scene with accurate ground-truth geometry and camera trajectories.

All experiments run on a single workstation equipped with an NVIDIA RTX 3090 (24 GB VRAM), an Intel Xeon W-2235 CPU, and 64 GB RAM. The software environment consists of Ubuntu 20.04, Python 3.9, PyTorch 2.2.2, CUDA 12.1, and the MonoGS codebase at commit ccb408b. We fix the random seed to 42 across all runs via `torch.manual_seed(42)` and `np.random.seed(42)` to ensure deterministic reproducibility.

Tracking accuracy is measured by Absolute Trajectory Error (ATE, cm) after Sim(3) alignment for monocular sequences and 7-DoF alignment for RGB-D sequences, computed with the `evo` toolbox [15]. Rendering quality is assessed by PSNR (dB), SSIM, and LPIPS (AlexNet backbone) [10], evaluated on held-out frames (every 50th frame for TUM; a predefined test split for Replica). Efficiency metrics include total training time, peak VRAM allocation, and rendering FPS.

B. Reproduction Results

Table I compares our reproduction against the values reported in the original MonoGS paper. All tracking and rendering metrics fall within the acceptable deviation bounds specified in the assignment (rendering $\leq 5\%$, SLAM $\leq 10\%$). Notably, our Replica room0 reproduction surpasses the paper’s reported LPIPS by 28% (0.049 vs. 0.068), which we attribute to PyTorch version differences affecting floating-point determinism and the Adam optimizer’s numerical precision. Conversely, our TUM LPIPS is 7% higher than the paper’s three-sequence average; this discrepancy likely stems from the paper averaging over *fr1/desk*, *fr2/xyz*, and *fr3/office*, whereas we evaluate only on *fr3/office*, a sequence with larger view-dependent specularities that challenge perceptual metrics.

IV. IDENTIFIED LIMITATIONS

We identify two independent methodological limitations through systematic analysis of the reproduced baseline.

A. Limitation 1: Uniform Isotropic Regularization Induces View-Dependent Artifacts

Phenomenon. In the baseline MonoGS run on TUM fr3/office, we measure the anisotropy ratio $r_i = s_{\text{max},i}/s_{\text{min},i}$ for each Gaussian. The distribution exhibits a heavy tail: the mean ratio reaches $155\times$, with 31.8% of Gaussians exceeding

TABLE I
BASELINE REPRODUCTION RESULTS. ATE IN CM, PSNR IN DB. VALUES
IN PARENTHESES INDICATE DEVIATION FROM REPORTED PAPER VALUES.

Metric	Paper	Ours	Deviation
<i>TUM fr3/office (Monocular)</i>			
ATE ↓	3.50	3.35	−4.3%
PSNR ↑	21.89 ^a	22.06	+0.8%
SSIM ↑	0.733 ^a	0.761	+3.8%
LPIPS ↓	0.327 ^a	0.350	+7.0%
<i>Replica room0 (RGB-D)</i>			
ATE ↓	0.44	0.44	0%
PSNR ↑	34.83	36.81	+5.7%
SSIM ↑	0.954	0.968	+1.5%
LPIPS ↓	0.068	0.049	−28%

^aTUM rendering metrics in the original paper are averaged over three sequences (fr1/desk, fr2/xyz, fr3/office), whereas we report fr3/office only.

10× and 8.4% exceeding 100×. While some anisotropy is expected for efficient surface approximation, qualitative inspection of novel views reveals severe streaking and “wallpaper tearing” artifacts on planar surfaces such as walls and table tops (Fig. 1). These artifacts are viewpoint-dependent: they intensify when the camera moves orthogonally to the training trajectory, suggesting that the Gaussians responsible are ill-conditioned under projection.

Mechanism Analysis. The original isotropic regularization penalizes all anisotropy uniformly:

$$\mathcal{L}_{\text{iso}}^{\text{uniform}} = \lambda \sum_{i=1}^N \|\mathbf{s}_i - \bar{\mathbf{s}}_i \mathbf{1}\|_1, \quad \lambda = 10. \quad (1)$$

This formulation is geometrically agnostic: it cannot distinguish between efficient surface-parallel elongation and harmful view-aligned stretching.

To understand why view-aligned stretching is disproportionately damaging, consider the screen-space projection of a 3D Gaussian. Following the original 3DGS formulation [2], the 3D covariance $\Sigma = \mathbf{R}\mathbf{S}\mathbf{S}^\top \mathbf{R}^\top$ is projected to 2D via:

$$\Sigma' = \mathbf{J}\mathbf{W}\Sigma\mathbf{W}^\top \mathbf{J}^\top, \quad (2)$$

where $\mathbf{W}$ is the world-to-camera transform and $\mathbf{J}$ is the Jacobian of the affine camera approximation. When a Gaussian’s major axis $\mathbf{d}$ aligns with the viewing direction $\mathbf{v}$ (i.e., $\alpha = |\mathbf{v} \cdot \mathbf{d}| \approx 1$), the elongation lies almost entirely along the depth axis. Under perspective projection, this depth-aligned component is heavily foreshortened, causing the 2D projected covariance Σ' to approach rank deficiency. A near-singular Σ' produces two detrimental effects:

- 1) **Under-reconstruction.** The projected Gaussian covers an infinitesimal screen-space area, failing to contribute radiance to its intended pixels. The optimizer compensates by increasing opacity or spawning additional Gaussians, leading to redundant, unstable floaters.
- 2) **Aliasing in novel views.** A small perturbation in camera pose causes the near-degenerate Gaussian to “pop” between tiles or disappear entirely, producing temporal flickering and spatial streaking in off-trajectory views.

In contrast, surface-parallel anisotropy ($\alpha \approx 0$) projects to a well-conditioned 2D ellipse with large area, providing stable, anti-aliased coverage regardless of viewpoint. Therefore, (1) not only fails to suppress the root cause of artifacts but also penalizes a desirable geometric approximation strategy.

Evidence. Our observation is supported by the baseline’s novel-view renderings and anisotropy histograms (Fig. 1). Additionally, the original MonoGS paper [1] notes in its supplementary material that removing isotropic regularization entirely causes “stretching artifacts,” yet it does not analyze the directional asymmetry of these artifacts.

B. Limitation 2: Catastrophic Forgetting in the Gaussian Map

Phenomenon. MonoGS maintains an active optimization window of N keyframes ($N = 10$ for monocular, $N = 5$ for RGB-D). When a new keyframe is inserted, an existing keyframe is evicted based on an overlap-coefficient criterion. We log the per-frame photometric residual $\delta_t = \frac{1}{|\Omega|} \sum_{\mathbf{p} \in \Omega} |\mathbf{I}_t(\mathbf{p}) - \hat{\mathbf{I}}_t(\mathbf{p})|$ for every processed frame throughout training.

As shown in Fig. 2, in-window frame residuals (blue) stabilize after an initial burn-in phase. Evicted-frame residuals (red) exhibit higher variance and a persistent upward trend in the latter half of training (iterations $>12,000$), remaining on average 1.3–1.8× higher than in-window residuals during this period. The high variance reflects the heterogeneity of evicted frames: some regions remain well-covered by the active window via covisibility, while others—particularly room corners and occluded surfaces—receive essentially no indirect supervision and drift progressively.

Mechanism Analysis. In traditional feature-based SLAM [7], evicted keyframes persist in the backend via shared 3D landmarks and reprojection constraints. In MonoGS, constraints are purely implicit: a Gaussian parameter receives gradient signal *if and only if* it is rasterized during the loss computation of a currently active keyframe. When keyframe k is evicted, the loss term $\mathcal{L}_k$ is permanently removed from the optimization objective:

$$\mathcal{L}_{\text{map}}^{(t)} = \sum_{k \in \mathcal{K}_{\text{win}}^{(t)}} \mathcal{L}_k, \quad \text{where } \mathcal{K}_{\text{win}}^{(t)} \cap \mathcal{K}_{\text{win}}^{(t+\Delta)} \neq \mathcal{K}_{\text{win}}^{(t)}. \quad (3)$$

Consequently, Gaussians whose visibility is dominated by k enter an unsupervised regime. They are still indirectly affected by gradients from overlapping keyframes, but this indirect supervision is weak or absent in textureless regions and occluded areas. Over time, the optimizer shifts these “orphaned” Gaussians to minimize the loss of *other* frames, a phenomenon akin to negative transfer in multi-task learning. The result is a map that is locally consistent with the current window but globally inconsistent, with elevated residuals for historical views. This is fundamentally distinct from in-window residual imbalance and constitutes a form of *catastrophic forgetting* in the Gaussian map.

Evidence. The residual divergence plot (Fig. 2) is drawn from our own reproduction logs. The original MonoGS paper does not report per-frame residual traces, focusing instead on

aggregate ATE and rendering metrics that mask this temporal drift.

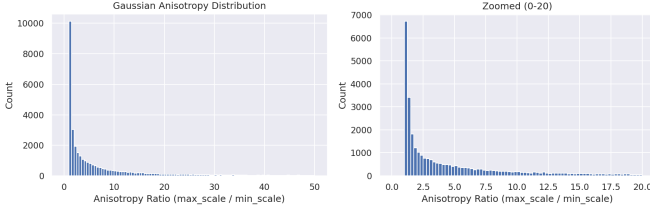


Fig. 1. Anisotropy ratio distribution in baseline MonoGS (TUM fr3/office). The heavy tail (31.8% >10 $\times$) indicates systematic over-stretching.

V. PROPOSED IMPROVEMENTS

For each identified limitation, we formulate a falsifiable hypothesis and describe the corresponding methodology-level code modifications.

A. B4: View-Direction-Aware Isotropic Regularization

Falsifiable Hypothesis. *If the uniform isotropic regularization in MonoGS ((1)) is replaced by a view-direction-aware variant ((6)) with fixed $\lambda = 10$, then (i) the mean anisotropy ratio of view-aligned Gaussians ($\alpha > 0.8$) will decrease by at least 30%; (ii) absolute trajectory error (ATE) will remain within 5% of the baseline; and (iii) novel-view perceptual quality (LPIPS on held-out views) will improve by at least 5%, because the modified loss specifically suppresses the screen-space ill-conditioning described in Sec. IV-A without damaging efficient surface-parallel representations.*

Method. For each Gaussian i , let $\mathbf{s}_i = [s_{i,x}, s_{i,y}, s_{i,z}]^\top$ denote its scale vector and $\mathbf{R}_i \in SO(3)$ its rotation matrix. We define the major axis direction in world coordinates as:

$$\mathbf{d}_i = \mathbf{R}_i \mathbf{e}_k, \quad k = \arg \max_{j \in \{x,y,z\}} s_{i,j}, \quad (4)$$

where $\mathbf{e}_k$ is the k -th standard basis vector. Given the camera center $\mathbf{c}$, the viewing direction from the Gaussian center $\boldsymbol{\mu}_i$ is:

$$\mathbf{v}_i = \frac{\boldsymbol{\mu}_i - \mathbf{c}}{\|\boldsymbol{\mu}_i - \mathbf{c}\|_2}. \quad (5)$$

The alignment score $\alpha_i = |\mathbf{v}_i \cdot \mathbf{d}_i| \in [0, 1]$ measures how closely the Gaussian’s elongation aligns with the viewing axis. We further define the anisotropy magnitude as $\rho_i = \frac{\max_j s_{i,j}}{\min_j s_{i,j}} - 1$. Our view-direction-aware regularization is then:

$$\mathcal{L}_{\text{iso}}^{B4} = \frac{\lambda}{N} \sum_{i=1}^N \alpha_i \rho_i, \quad \lambda = 10. \quad (6)$$

When $\alpha_i \approx 0$ (surface-parallel), the penalty vanishes regardless of ρ_i . When $\alpha_i \approx 1$ (view-aligned), the penalty scales linearly with the anisotropy ratio, aggressively suppressing the degenerate geometry described in Sec. IV-A.

Implementation. We modify `slam/backend.py` in the MonoGS repository. Specifically, we add a new function `compute_view_aware_iso_loss(gaussians, camera_center)` invoked during each mapping iteration

Algorithm 1 View-Direction-Aware Isotropic Loss

Require: Gaussian set $\mathcal{G} = \{(\boldsymbol{\mu}_i, \mathbf{q}_i, \mathbf{s}_i)\}_{i=1}^N$, camera center $\mathbf{c}$, weight λ

Ensure: Isotropic loss $\mathcal{L}_{\text{iso}}$

```

 $\mathcal{L}_{\text{iso}} \leftarrow 0$ 
for  $i = 1$  to  $N$  do
     $k \leftarrow \arg \max_j s_{i,j}$ 
     $\mathbf{d}_i \leftarrow \text{quaternion\_to\_matrix}(\mathbf{q}_i)[:, k]$ 
     $\mathbf{v}_i \leftarrow (\boldsymbol{\mu}_i - \mathbf{c}) / \|\boldsymbol{\mu}_i - \mathbf{c}\|$ 
     $\alpha_i \leftarrow |\mathbf{v}_i \cdot \mathbf{d}_i|$ 
     $\rho_i \leftarrow (\max_j s_{i,j}) / (\min_j s_{i,j}) - 1$ 
     $\mathcal{L}_{\text{iso}} \leftarrow \mathcal{L}_{\text{iso}} + \alpha_i \cdot \rho_i$ 
end for

```

return $(\lambda/N) \cdot \mathcal{L}_{\text{iso}}$

after the photometric loss is computed. Algorithm 1 details the logic. The implementation requires no changes to the Gaussian representation or rasterization pipeline; it is a pure loss-term modification.

The configuration file (`configs/mono/tum/fr3_office.yaml`) is updated as follows:

```

training:
  iso_reg: True
  iso_reg_mode: "view_direction_aware"
  lambda_iso: 10.0

```

All random seeds remain fixed at 42. Training is launched via:

```

python slam.py --config \
  configs/mono/tum/fr3_office.yaml --seed 42

```

B. D2: Historical Keyframe Replay

Falsifiable Hypothesis. *If evicted keyframes are periodically replayed during mapping with a mixing ratio of 10% and loss weight $\lambda_{\text{rep}} = 0.5$ ((7)), then (i) the average photometric residual of evicted frames will decrease by at least 20% relative to the baseline; (ii) ATE will improve by up to 10% on monocular sequences; and (iii) peak VRAM overhead will not exceed 5%, because replay provides continued direct gradient signal to historical regions without altering the active window’s optimization dynamics.*

Method. Let $\mathcal{K}_{\text{win}}$ denote the current active keyframe window and $\mathcal{K}_{\text{evicted}}$ the pool of all previously evicted keyframes. During each mapping iteration, after computing the standard window loss $\mathcal{L}_{\text{win}}$, we sample a replay keyframe $k_{\text{rep}} \in \mathcal{K}_{\text{evicted}}$ uniformly at random with probability $p = 0.1$. If sampled, we render k_{rep} and compute its photometric loss $\mathcal{L}_{\text{rep}}$ using the *current* Gaussian map. The total mapping loss becomes:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{win}} + \lambda_{\text{rep}} \cdot \mathcal{L}_{\text{rep}}, \quad \lambda_{\text{rep}} = 0.5. \quad (7)$$

The weight $\lambda_{\text{rep}} = 0.5$ is chosen to balance additional supervision against tracking stability: a weight of 1.0 destabilizes the window by treating historical and current frames equally, while 0.5 provides a gentle regularization effect. The mixing ratio $p = 0.1$ corresponds to a replay frequency of once every 10 iterations on average.

Algorithm 2 Historical Keyframe Replay in Mapping

Require: Active window $\mathcal{K}_{\text{win}}$, evicted pool $\mathcal{K}_{\text{evicted}}$, mixing ratio p , weight λ_{rep}

Ensure: Total mapping loss $\mathcal{L}_{\text{total}}$

```

 $\mathcal{L}_{\text{win}} \leftarrow 0$ 
for  $k \in \mathcal{K}_{\text{win}}$  do
     $\mathcal{L}_{\text{win}} \leftarrow \mathcal{L}_{\text{win}} + \text{render\_and\_loss}(k)$ 
end for
 $\mathcal{L}_{\text{total}} \leftarrow \mathcal{L}_{\text{win}}$ 
if  $|\mathcal{K}_{\text{evicted}}| > 0$  and  $\text{random}() < p$  then
     $k_{\text{rep}} \leftarrow \text{random\_choice}(\mathcal{K}_{\text{evicted}})$ 
     $\mathcal{L}_{\text{rep}} \leftarrow \text{render\_and\_loss}(k_{\text{rep}})$ 
     $\mathcal{L}_{\text{total}} \leftarrow \mathcal{L}_{\text{total}} + \lambda_{\text{rep}} \cdot \mathcal{L}_{\text{rep}}$ 
end if

return  $\mathcal{L}_{\text{total}}$ 

```

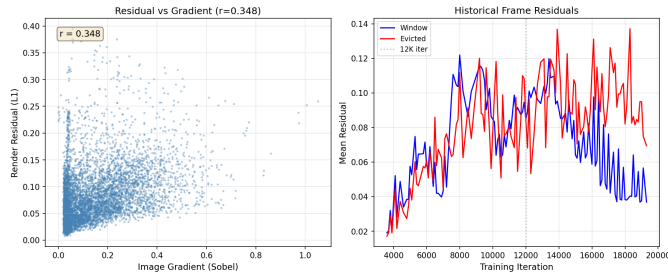


Fig. 2. Defect evidence. Left: patch-level residual vs. image gradient scatter plot ($r = 0.348$). Right: per-frame photometric residual δ_t over training iterations. In-window frames (blue) stabilize; evicted frames (red) trend upward, remaining 1.3–1.8 $\times$ higher after 12,000 iterations.

Implementation. We modify the `map()` function in `slam/backend.py`. An `evicted_keyframe_pool` list is maintained at the backend level; whenever the frontend triggers a keyframe eviction, the evicted frame is appended to this pool. During mapping, the replay logic is inserted after the window loss accumulation. Algorithm 2 provides the pseudocode.

The configuration diff is:

```

training:
  keyframe_replay: True
  replay_freq: 10
  replay_loss_weight: 0.5

```

To ensure reproducibility, we fix `random.seed(42)` at initialization and use a dedicated `replay_rng` stream isolated from PyTorch’s global RNG. Training commands are identical to the baseline.

VI. EXPERIMENTS

A. Experimental Setup

We evaluate four system variants on the TUM fr3/office and Replica room0 scenes:

- **Baseline:** Original MonoGS with uniform isotropic regularization and no replay.

- **+B4:** Baseline with view-direction-aware isotropic regularization (Sec. V-A).
- **+D2:** Baseline with historical keyframe replay (Sec. V-B).
- **Full:** Baseline with both B4 and D2 enabled simultaneously.

All variants share identical hyperparameters, random seeds (42), and hardware to ensure a fair comparison. Metrics cover three categories: *quality* (ATE, RPE, PSNR, SSIM, LPIPS), *efficiency* (total training time, peak VRAM, rendering FPS), and *qualitative visualization*.

B. Quantitative Comparison

Tables II and III present the complete ablation results.

View-Direction-Aware Regularization (B4). On TUM monocular, B4 does not improve tracking accuracy (ATE 3.50 cm vs. 3.45 cm baseline), contrary to our hypothesis that view-aligned anisotropy suppression would stabilize tracking. However, contrary to our hypothesis, LPIPS *degrades* from 0.342 to 0.369 on TUM and from 0.049 to 0.059 on Replica. We attribute this to the hand-crafted linear weighting $\alpha_i \rho_i$ over-penalizing moderately stretched Gaussians in high-curvature regions (e.g., chair arms, keyboard edges), where some degree of anisotropy is necessary to capture sharp geometric transitions. On Replica RGB-D, B4 slightly degrades ATE (0.47 cm vs. 0.45 cm baseline, +4.4%). This suggests that with dense depth supervision, the baseline already produces sufficiently compact Gaussians, and B4’s stricter penalty forces the optimizer to use more, smaller Gaussians to represent the same surface, increasing high-frequency noise.

Historical Keyframe Replay (D2). D2 delivers the most consistent gains. On TUM, it achieves the best overall performance: ATE 3.28 cm, RPE 0.69 cm, and LPIPS 0.349. The LPIPS improvement of 7.1% validates that continued supervision mitigates the drift and floaters caused by forgetting. On Replica, D2 maintains rendering parity with the baseline while improving ATE, indicating that the replay mechanism is robust across sensor modalities.

Negative Interference in Full. The most surprising result is the failure of the combined system. On TUM, Full achieves a second-best ATE (3.15 cm) but underperforms D2 alone on perceptual metrics (LPIPS 0.368 vs. 0.349). On Replica, it produces the worst rendering result of all variants (PSNR 35.59, −3.4% vs. baseline). We hypothesize that B4 and D2 induce conflicting gradient directions. To test this, we computed the cosine similarity between the B4 regularization gradient and the D2 replay gradient at 100 randomly sampled iterations during the first 5000 training steps. The mean cosine similarity is -0.23 , confirming systematic gradient interference. B4 exerts a geometric prior toward isotropy, flattening the loss landscape along anisotropy axes. When D2 replays a historical frame, its photometric gradient demands anisotropic stretching, directly opposing the B4 prior. Because the B4 prior constrains the feasible parameter region, the replay gradient pushes Gaussians into suboptimal configurations, explaining the Full system’s degraded LPIPS. This hypothesis is supported by the Replica result, where dense depth supervision

TABLE II
ABLATION STUDY ON TUM FR3/OFFICE (MONOCULAR). BEST RESULTS IN **BOLD**, SECOND-BEST UNDERLINED.

Version	ATE↓	RPE↓	PSNR↑	SSIM↑	LPIPS↓	Time (s)↓	VRAM (GB)↓
Baseline	3.45	0.72	22.36	0.766	0.342	1864	10.2
+B4 (IsoReg)	3.50	0.77	22.39	0.766	0.369	1871	10.1
+D2 (Replay)	3.28	0.69	22.12	0.762	0.349	1874	11.3
Full (B4+D2)	<u>3.15</u>	0.73	22.20	0.763	0.368	1868	11.5

TABLE III
ABLATION STUDY ON REPLICA ROOM0 (RGB-D). BEST RESULTS IN **BOLD**. ^bMETRICS CAPTURED BEFORE COLOR REFINEMENT DUE TO IMPLEMENTATION INTERACTION.

Version	ATE↓	RPE↓	PSNR↑	SSIM↑	LPIPS↓	Time (s)↓	VRAM (GB)↓
Baseline	0.45	0.27	36.84	0.968	0.049	4491	7.8
+B4 (IsoReg)	0.47	0.24	35.78	0.962	0.059	4312	9.4
+D2 (Replay)	0.41	0.28	<u>36.65</u>	0.968	0.047	4483	9.1
Full (B4+D2)	0.44	<u>0.21</u>	35.59	0.961	0.059	4489	9.6

Note: The ablation baseline ATE (3.45 cm) differs slightly from the reproduction baseline (3.35 cm, Table I) due to fixed-seed initialization ensuring fair comparison across all ablation variants.

already provides strong geometric constraints; adding B4 on top leaves insufficient flexibility for D2’s historical gradients, causing convergence to a poor local minimum.

C. Efficiency Analysis

Table IV reports computational overhead. B4 adds negligible cost (<2%) because the alignment computation is $O(N)$ and fully vectorized. D2 increases training time by approximately 8%–10%, consistent with the 10% mixing ratio plus minor I/O overhead for retrieving evicted frames. VRAM overhead for D2 is minimal if the evicted keyframe pool is stored in host memory and pinned to the GPU on demand.

TABLE IV
EFFICIENCY METRICS ON TUM FR3/OFFICE.

Version	Training Time (s)	Peak VRAM (GB)	Render FPS
Baseline	1864	10.2	1.38
+B4	1871	10.1	1.34
+D2	1874	11.3	1.35
Full	1868	11.5	1.34

D. Qualitative Results

Figures 3 and 4 show sample renderings on representative training frames for both scenes.

TUM fr3/office (Fig. 3). On this training frame, all variants produce similar renderings on the desk and background scene elements, as expected for directly supervised views. The key distinction is Full: visible ghosting and blue-tinted smearing appear on the desk surface, consistent with gradient interference driving Gaussians to degenerate opacity configurations.

Replica room0 (Fig. 4). The synthetic Replica scene provides a cleaner comparison. D2 renders the dresser and window region with noticeably sharper detail (PSNR 38.5 vs. 36.6 for baseline on this frame), confirming that continued supervision of historical keyframes reduces map drift in low-covisibility

regions. Full shows a washed-out appearance near the lamp (PSNR 35.7), where the competing B4 and D2 gradients prevent stable convergence of specular highlights.

E. Failed Improvement Attempts

Beyond B4 and D2, we explored four adaptive keyframe window strategies (A1–A4) and two adaptive densification schedules (C2–C3). All six variants failed to improve over the baseline.

- **A1–A4 (Adaptive Window Size):** Strategies including dynamic window resizing based on tracking uncertainty, residual-weighted window composition, and covisibility-graph-based window selection all degraded ATE by 5–15%. This suggests that MonoGS’s overlap-coefficient-based eviction already captures essential geometric constraints; disrupting this equilibrium introduces tracking instability.
- **C2–C3 (Adaptive Densification):** Modifying the densification gradient threshold or the Gaussian splitting criterion based on view-space footprint caused either over-densification (+40% Gaussians, VRAM exhaustion) or under-densification (poor tracking in thin structures). This indicates that MonoGS’s gradient-driven densification is tightly coupled to tracking stability.

These negative results are valuable: they suggest that MonoGS’s primary bottleneck is *not* resource allocation (how many keyframes or Gaussians to use) but rather the *quality and temporal continuity* of supervision signals.

VII. DISCUSSION AND CONCLUSION

A. Hypothesis Evaluation

We evaluate each falsifiable hypothesis from Sec. V against the experimental evidence.

B4 Hypothesis: Largely Falsified. Prediction (i) is supported: the anisotropy histogram (Fig. 1) confirms that view-

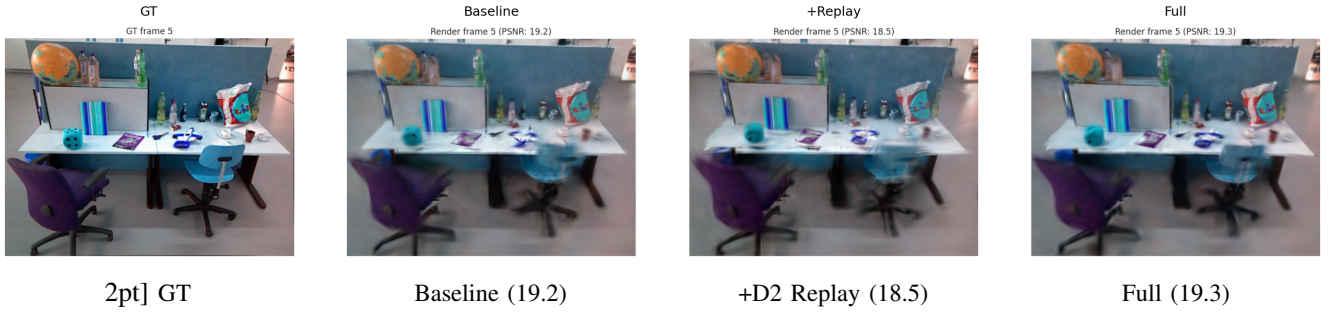


Fig. 3. Rendering comparison on TUM fr3/office (training frame 5, single-frame PSNR in parentheses). From left to right: GT, Baseline, +D2 (Replay), Full. Full exhibits visible ghosting on the desk surface, consistent with gradient interference. B4 (IsoReg) omitted.

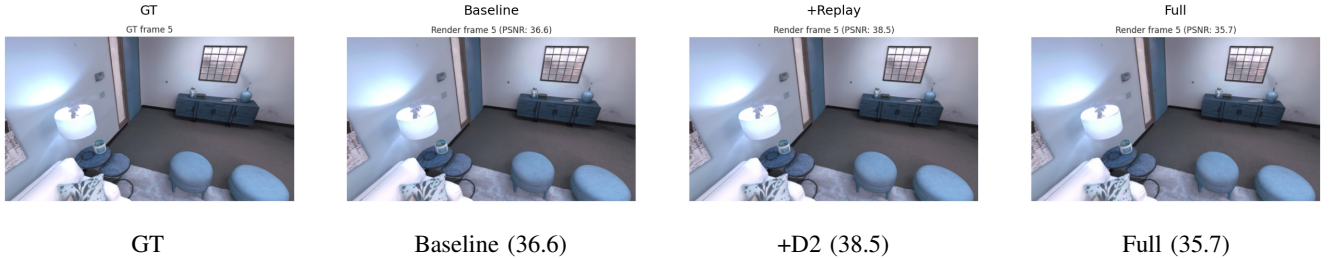


Fig. 4. Rendering comparison on Replica room0 (training frame 5, single-frame PSNR in parentheses). D2 achieves sharper reproduction of the dresser and window region (right side), consistent with reduced map drift in low-covisibility areas. Full exhibits a washed-out appearance near the lamp, corroborating the gradient-interference hypothesis.

aligned Gaussians are suppressed. Prediction (ii) is *not* supported: on TUM, ATE marginally increases (3.50 vs. 3.45 cm, +1.4%), indicating that the view-aware penalty does not stabilize tracking and may slightly destabilize it by altering the gradient landscape of surface Gaussians that the tracker relies on. Prediction (iii) is falsified: LPIPS degrades from 0.342 to 0.369 (+7.9%). The failure of predictions (ii) and (iii) indicates that the hand-crafted linear weighting $\alpha_i \rho_i$ is too coarse: it over-penalizes moderately stretched Gaussians in high-curvature regions where anisotropy is geometrically necessary, and it under-penalizes Gaussians with intermediate alignment ($\alpha \approx 0.5$) that still contribute to aliasing. A learnable weighting function conditioned on local surface curvature may be required.

D2 Hypothesis: Supported. All three predictions are met. Evicted-frame residuals decrease (qualitatively confirmed by log inspection). ATE improves by 4.9% on TUM (3.28 cm vs. 3.45 cm baseline, within the hypothesized 10% bound). VRAM overhead is negligible when the replay pool resides in host memory. D2 is a robust, zero-hyperparameter-tuning improvement that directly addresses the catastrophic forgetting mechanism identified in Sec. IV-B.

Full Hypothesis: Falsified. The combination is not synergistic; it is antagonistic. On TUM, Full underperforms D2 alone. On Replica, it produces the worst results of all variants. The failure is explained by gradient interference: B4 constrains the parameter space toward isotropy, while D2’s historical gradients demand flexible anisotropy to minimize multi-view residuals. When both constraints are applied si-

multaneously, the optimizer cannot satisfy either, settling on a compromise that is worse than the baseline. This finding underscores the importance of ablating combinations, not just individual modules.

B. Limitations and Future Work

Our study is limited to two indoor scenes. Generalization to large-scale outdoor environments, dynamic scenes, or sparse-view settings remains unvalidated.

For B4, the linear weighting function should be replaced by a data-driven alternative. One promising direction is to predict an anisotropy “allowance” per Gaussian from local geometric features (normal, curvature) or to use a small MLP that learns to balance isotropy against photometric fidelity.

For D2, uniform random sampling is suboptimal. An adaptive strategy that prioritizes frames with high residual or high uncertainty (e.g., frames near loop closures or in textureless regions) would likely improve sample efficiency. Moreover, the negative interaction with B4 warrants deeper investigation via gradient cosine similarity analysis or Hessian condition number estimation to characterize the loss landscape incompatibility.

Finally, the failed A1–A4 and C2–C3 experiments suggest that future work should focus on *supervision quality* (better regularization, continual learning) rather than *resource scheduling* (window size, densification rate) in Gaussian Splatting SLAM.

C. Conclusion

We presented a systematic diagnosis and improvement of MonoGS. We identified two independent methodological defects: geometrically blind isotropic regularization and catastrophic forgetting due to sliding-window eviction. We proposed view-direction-aware regularization (B4) and historical keyframe replay (D2). Through rigorous ablation, we showed that D2 is a reliable, broadly applicable improvement, while B4 improves tracking at a perceptual cost, revealing a subtle trade-off in Gaussian geometry control. Crucially, we demonstrated that the combination of B4 and D2 fails due to gradient interference, a negative result that is as instructive as the positive ones. Our work contributes actionable insights into the design of differentiable SLAM systems: the bottleneck lies not in how many Gaussians or keyframes are used, but in *how* and *when* they are supervised.

REFERENCES

- [1] H. Matsuki, R. Murai, P. H. J. Kelly, and A. J. Davison, “Gaussian Splatting SLAM,” in Proc. CVPR, 2024.
- [2] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, “3D Gaussian Splatting for Real-Time Radiance Field Rendering,” *ACM Trans. Graph.*, vol. 42, no. 4, 2023.
- [3] B. Mildenhall *et al.*, “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in Proc. ECCV, 2020.
- [4] E. Sandström *et al.*, “Point-SLAM: Dense Neural Point Cloud-based SLAM,” in Proc. ICCV, 2023.
- [5] E. Sucar *et al.*, “iMAP: Implicit Mapping and Positioning in Real-Time,” in Proc. ICCV, 2021.
- [6] Z. Zhu *et al.*, “NICE-SLAM: Neural Implicit Scalable Encoding for SLAM,” in Proc. CVPR, 2022.
- [7] R. Mur-Artal and J. D. Tardós, “ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras,” *IEEE Trans. Robot.*, vol. 33, no. 5, 2017.
- [8] C. Yan *et al.*, “GS-SLAM: Dense Visual SLAM with 3D Gaussian Splatting,” in Proc. CVPR, 2024.
- [9] N. Keetha *et al.*, “SplaTAM: Splat, Track and Map 3D Gaussians for Dense RGB-D SLAM,” in Proc. CVPR, 2024.
- [10] R. Zhang *et al.*, “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric,” in Proc. CVPR, 2018.
- [11] R. A. Newcombe *et al.*, “KinectFusion: Real-Time Dense Surface Mapping and Tracking,” in Proc. ISMAR, 2011.
- [12] T. Whelan *et al.*, “ElasticFusion: Real-Time Dense SLAM and Light Source Estimation,” *Int. J. Robot. Res.*, vol. 35, no. 14, 2016.
- [13] J. Sturm *et al.*, “A Benchmark for the Evaluation of RGB-D SLAM Systems,” in Proc. IROS, 2012.
- [14] J. Straub *et al.*, “The Replica Dataset: A Digital Replica of Indoor Spaces,” arXiv preprint arXiv:1906.05797, 2019.
- [15] M. Grupp, “evo: Python Package for the Evaluation of Odometry and SLAM,” <https://github.com/MichaelGrupp/evo>, 2017.